

Report 7 - Reduce

October 3, 2026

0.1 Grayscale

```
@c.jit
def grayscale(src, dst):
    tid = c.threadIdx.x + c.blockIdx.x * c.blockDim.x
    tidy = c.threadIdx.y + c.blockIdx.y * c.blockDim.y

    if tidy < src.shape[0] and tid < src.shape[1]:
        g = int((int(src[tidy, tid, 0]) + int(src[tidy, tid, 1]) + int(src[tidy, tid, 2])) / 3)
        dst[tidy, tid, 0] = dst[tidy, tid, 1] = dst[tidy, tid, 2] = g
```

0.2 Reduce - compute min and max values

```
@c.jit
def reduce(dst, global_min, global_max):
    tid = c.threadIdx.x + c.blockIdx.x * c.blockDim.x
    tidy = c.threadIdx.y + c.blockIdx.y * c.blockDim.y

    localtid = c.threadIdx.x
    localtidy = c.threadIdx.y
```

I hardcoded the cuda shared array values as it was causing consistent GPU crashes in the past.

```
min_cache = c.shared.array((16, 16), numba.int32)
max_cache = c.shared.array((16, 16), numba.int32)

# copy local block from dst to cache (in shared memory)
if tidy < dst.shape[0] and tid < dst.shape[1]:
    min_cache[localtidy, localtid] = dst[tidy, tid, 0]
    max_cache[localtidy, localtid] = dst[tidy, tid, 0]
else:
    min_cache[localtidy, localtid] = 255
    max_cache[localtidy, localtid] = 0

c.syncthreads()
# reduction in cache
sy = int(c.blockDim.y / 2)
sx = int(c.blockDim.x / 2)
```

The loop to move the min and max to index 0 of the cache

```
while sy > 0 and sx > 0:
    if (localtidy < sy) and (localtid < sx):
        min_cache[localtidy, localtid] = min(min_cache[localtidy, localtid], min_cache[localtidy, 0])
        max_cache[localtidy, localtid] = max(max_cache[localtidy, localtid], max_cache[localtidy, 0])
    sy -= 1
    sx -= 1
```

```

c.syncthreads()
sx = sx // 2
sy = sy // 2

```

These lines are to pass the values as global variables, i wasn't sure how to do it so I had to ask GenAI in english

```

c.atomic.min(global_min, 0, min_cache[0, 0])
c.atomic.max(global_max, 0, max_cache[0, 0])

```

0.3 Recalculating intensity

```

@c.jit
def recalculateIntensity(src, dst, g_min, g_max):
    tidx = c.threadIdx.x + c.blockIdx.x * c.blockDim.x
    tidy = c.threadIdx.y + c.blockIdx.y * c.blockDim.y
    local_min = g_min[0]
    local_max = g_max[0]
    if tidy < dst.shape[0] and tidx < dst.shape[1]:
        if local_max - local_min > 0:
            g = int((src[tidy, tidx, 0] - local_min) * 255 / (local_max - local_min))
            dst[tidy, tidx, 0] = dst[tidy, tidx, 1] = dst[tidy, tidx, 2] = g

```

0.4 Conclusion

I had to manually lower the contrast of my image to see an actual change, so here are the 3 steps illustrated:

